

Википедия

DSA

Материал из Википедии — свободной энциклопедии

DSA (англ. *Digital Signature Algorithm* — *алгоритм цифровой подписи*) — криптографический алгоритм с использованием закрытого ключа (из пары ключей: <открытый; закрытый>) для создания электронной подписи, но не для шифрования (в отличие от RSA и схемы Эль-Гамала). Подпись создается секретно (закрытым ключом), но может быть публично проверена (открытым ключом). Это означает, что только один субъект может создать подпись сообщения, но любой может проверить её корректность. Алгоритм основан на вычислительной сложности взятия логарифмов в конечных полях.

DSA, Digital Signature Algorithm	
Создатель	NIST
Создан	1991 год
Опубликован	1998 год
Размер ключа	закрытый: 160-256 бит, открытый: 1024-3072 бит
Размер подписи	два числа по 160-256 бит

Алгоритм был предложен Национальным институтом стандартов и технологий (США) в августе 1991 и является запатентованным^[1] (автор патента — David W. Kravitz), НИСТ сделал этот патент доступным для использования без лицензионных отчислений. DSA является частью **DSS** (англ. *Digital Signature Standard* — стандарт цифровой подписи), впервые опубликованного 15 декабря 1998 (документ FIPS-186^[2] (англ. *Federal Information Processing Standards* — федеральные стандарты обработки информации)). Стандарт несколько раз обновлялся^{[3][4]}, последняя версия FIPS-186-4^[5]. (июль 2013).

Содержание

- Описание алгоритма
- Параметры схемы цифровой подписи
 - Открытый и секретный ключи
- Подпись сообщения
- Проверка подписи
- Корректность схемы
- Пример работы
 - Генерация параметров
 - Создание ключей
 - Подпись сообщения
 - Проверка подписи
- Аналоги
- Криптографическая стойкость
 - Предсказуемость случайного параметра
 - Existential forgery

[Восстановление ключа](#)[Подделка подписи](#)

Система проверки реализации алгоритма

[Генерация простых чисел](#)[Генерация случайных чисел](#)[Примечания](#)

Литература

[Стандарты и патенты](#)[Статьи](#)

Описание алгоритма

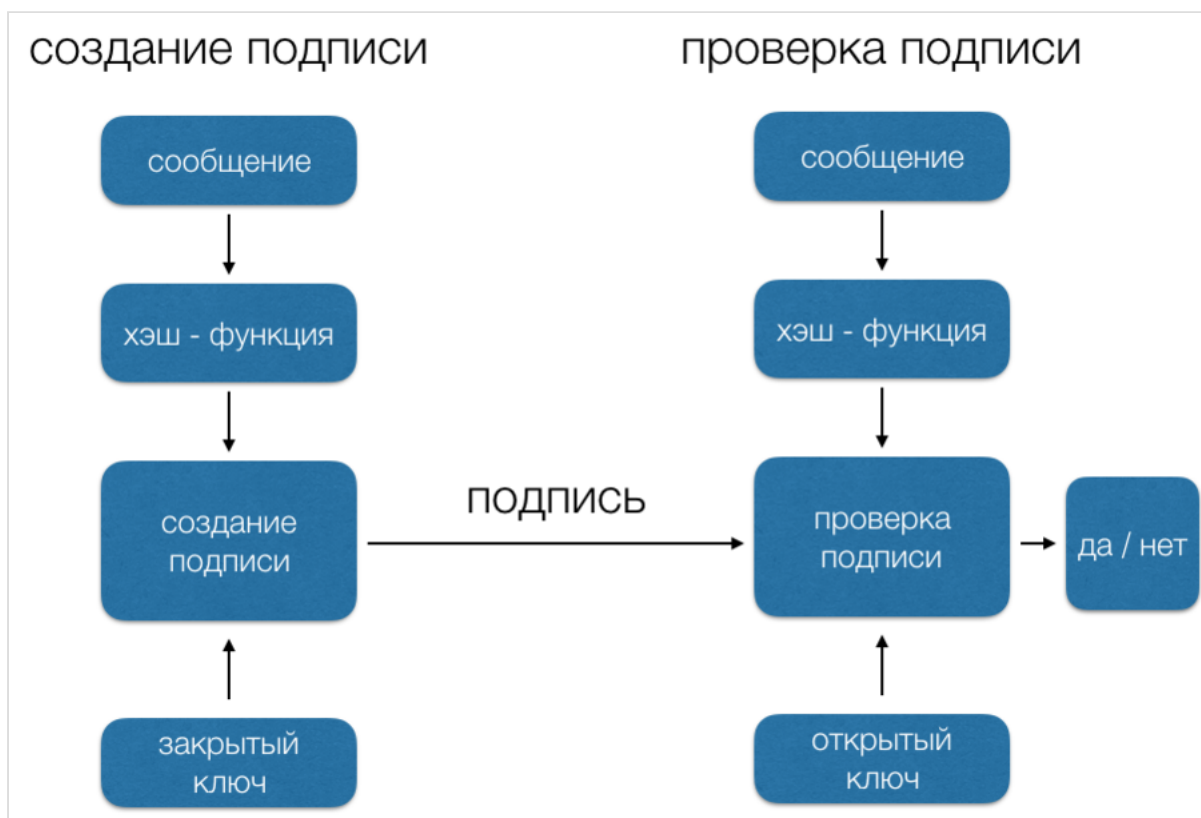


иллюстрация работы DSA

DSA включает в себя два алгоритма (S, V): для создания подписи сообщения (S) и для ее проверки (V).

Оба алгоритма вначале вычисляют хеш сообщения, используя криптографическую хеш-функцию. Алгоритм S использует хеш и секретный ключ для создания подписи, алгоритм V использует хеш сообщения, подпись и открытый ключ для проверки подписи.

Стоит подчеркнуть, что фактически подписывается не сообщение (произвольной длины), а его хеш (160 - 256 бит), поэтому неизбежны коллизии и одна подпись, вообще говоря, действительна для нескольких сообщений с одинаковым хешем. Поэтому выбор достаточно "хорошей" хеш-функции очень важен для всей системы в целом. В первой версии стандарта использовалась хеш-функция SHA-1^{[6][2]} (англ. Secure Hash Algorithm - *безопасный алгоритм хеширования*), в последней версии также можно использовать любой алгоритм семейства SHA-2^{[6][5]}. В августе 2015 был опубликован FIPS-202^[7], описывающий новую хеш-функцию SHA-3. Но на сегодняшний день она не включена в стандарт DSS^[5].

Для работы системы требуется база соответствия между реальными реквизитами автора (это может быть как частное лицо, так и организация) и открытыми ключами, а также всеми необходимыми параметрами схемы цифровой подписи (хеш-функция, простые числа). Например, подобной базой может служить центр сертификации.

Параметры схемы цифровой подписи

Для построения системы цифровой подписи нужно выполнить следующие шаги:

1. Выбор криптографической хеш-функции $H(x)$.
2. Выбор простого числа q , размерность которого N в битах совпадает с размерностью в битах значений хеш-функции $H(x)$.
3. Выбор простого числа p , такого, что $(p-1)$ делится на q . Битовая длина p обозначается L .
4. Выбор числа g ($g \neq 1$) такого, что его мультипликативный порядок по модулю p равен q .

Для его вычисления можно воспользоваться формулой $g = h^{(p-1)/q} \bmod p$, где h — некоторое произвольное число, $h \in (1; p-1)$ такое, что $g \neq 1$. В большинстве случаев значение $h = 2$ удовлетворяет этому требованию.^[5]

Как упомянуто выше, первоочередным параметром схемы цифровой подписи является используемая криптографическая хеш-функция, необходимая для преобразования текста сообщения в число, для которого и вычисляется подпись. Важной характеристикой этой функции является битовая длина выходной последовательности, обозначаемая далее N . В первой версии стандарта DSS рекомендована функция SHA-1^[2] и, соответственно, битовая длина подписываемого числа 160 бит. Сейчас SHA-1 уже не является достаточно безопасной^{[8][9]}. В стандарте указаны следующие возможные пары значений чисел L и N :

1. $L = 1024$, $N = 160$
2. $L = 2048$, $N = 224$
3. $L = 2048$, $N = 256$
4. $L = 3072$, $N = 256$

В соответствии с этим стандартом рекомендованы хеш-функции семейства SHA-2. Правительственные организации США должны использовать один из первых трех вариантов, центры сертификации должны использовать пару, которая равна или превосходит пару, используемую подписчиками^[5]. Проектирующий систему может выбрать любую допустимую хеш-функцию. Поэтому далее не будет заостряться внимание на использовании конкретной хеш-функции.

Стойкость криптосистемы на основе DSA не превосходит стойкость используемой хеш-функции и стойкость пары (L, N) , чья стойкость не больше стойкости каждого из чисел по отдельности. Также важно учитывать, как долго система должна оставаться безопасной. В данный момент для систем, которые должны быть стойкими до 2010 (2030) года, рекомендуется длина в 2048 (3072) бита.^{[5][10]}

Открытый и секретный ключи

1. Секретный ключ представляет собой число $x \in (0, q)$
2. Открытый ключ вычисляется по формуле $y = g^x \bmod p$

Открытыми параметрами являются числа (p, q, g, y) . Закрытый параметр только один — число x . При этом числа (p, q, g) могут быть общими для группы пользователей, а числа x и y являются соответственно закрытым и открытым ключами конкретного пользователя. При подписывании сообщения используются секретные числа x и k , причем число k должно выбираться случайным образом (на практике псевдослучайным) при вычислении подписи каждого следующего сообщения.

Поскольку (p, q, g) могут быть использованы для нескольких пользователей, на практике часто делят пользователей по некоторым критериям на группы с одинаковыми (p, q, g) .

Подпись сообщения

Подпись сообщения выполняется по следующему алгоритму^[5]:

1. Выбор случайного числа $k \in (0, q)$
2. Вычисление $r = (g^k \bmod p) \bmod q$
3. Выбор другого k , если $r = 0$
4. Вычисление $s = k^{-1}(H(m) + x \cdot r) \bmod q$
5. Выбор другого k , если $s = 0$
6. Подписью является пара (r, s) общей длины $2N$

Вычислительно сложные операции это возведение в степень по модулю (вычисление $g^k \bmod p$), для которого существуют быстрые алгоритмы, вычисление хеша $H(x)$, где сложность зависит от выбранного алгоритма хеширования и размера входного сообщения, и нахождение обратного элемента $k^{-1} \bmod q$ используя, например, расширенный алгоритм Евклида или малую теорему Ферма в виде $k^{-1} \bmod q = k^{q-2} \bmod q$.

Проверка подписи

Проверка подписи выполняется по алгоритму^[5]:

1. Вычисление $w = s^{-1} \bmod q$
2. Вычисление $u_1 = H(m) \cdot w \bmod q$
3. Вычисление $u_2 = r \cdot w \bmod q$
4. Вычисление $v = (g^{u_1} \cdot y^{u_2} \bmod p) \bmod q$
5. Подпись верна, если $v = r$

При проверке вычислительно сложные операции это два возведения в степень $g^{u_1} y^{u_2}$, вычисление хеша $H(x)$ и нахождение обратного элемента $s^{-1} \bmod q$.

Корректность схемы

Данная схема цифровой подписи корректна в той степени, что желающий проверить подлинность подписи всегда получит положительный результат в случае подлинности. Покажем это:

Во-первых, если $g = h^{(p-1)/q} \bmod p$, то из этого по Малой теореме Ферма следует $g^q = h^{p-1} = 1 \bmod p$. Поскольку $g > 1$ и q простое число, то g должно иметь мультипликативный порядок q по модулю p .

Для подписи сообщения вычисляется

$$s = k^{-1}(H(m) + x \cdot r) \mod q.$$

Из этого следует

$$k = H(m) \cdot s^{-1} + x \cdot r \cdot s^{-1} = H(m) \cdot w + x \cdot r \cdot w \mod q.$$

Так как g имеет порядок q , получим

$$g^k = g^{H(m) \cdot w \mod q} g^{x \cdot r \cdot w \mod q} = g^{H(m) \cdot w \mod q} y^{r \cdot w \mod q} = g^{u_1} y^{u_2} \mod p.$$

Наконец, корректность схемы DSA следует из

$$r = (g^k \mod p) \mod q = (g^{u_1} y^{u_2} \mod p) \mod q = v. \text{[5]}$$

Пример работы

Приведем пример работы алгоритма для небольших чисел. Пусть значение хеш - функции нашего сообщения $H = 9$.

Генерация параметров

1. $H = 9_{10} = 1001_2$
2. длина хеша 4, значит можно выбрать $q = 11_{10} = 1011_2$
3. выберем $p = 23$, так как $23 - 1 = 22 = 2 * q$
4. выберем $g = 2^2 = 4$

Создание ключей

1. в качестве секретного ключа выберем $x = 7$
2. тогда открытый ключ
 $y = g^x \mod p = 4^7 \mod 23 = 16384 \mod 23 = (712 \cdot 23 + 8) \mod 23 = 8$

Подпись сообщения

1. выберем $k = 3$
2. тогда $r = (g^k \mod p) \mod q = (4^3 \mod 23) \mod 11 = 7$
3. $r \neq 0$, идем дальше
4. $s = k^{-1}(H(m) + x \cdot r) \mod q = 4(9 + 7 \cdot 7) \mod 11 = 1$, где учтено, что $3^{-1} \mod 11 = 4$
5. $s \neq 0$, идем дальше
6. подписью является пара чисел $(r, s) = (7, 1)$

Проверка подписи

1. $w = s^{-1} \mod q = 1^{-1} \mod 11 = 1$
2. $u_1 = H(m) \cdot w \mod q = 9 \cdot 1 \mod 11 = 9$

$$3. u_2 = r \cdot w \bmod q = 7 \cdot 1 \bmod 11 = 7$$

$$4. v = (g^{u_1} \cdot y^{u_2} \bmod p) \bmod q = (4^9 \cdot 8^7 \bmod 23) \bmod 11 = 7$$

5. получили, что $v = r$, значит подпись верна.

Аналоги

Алгоритм DSA основывается на трудности вычисления дискретных логарифмов и является модификацией классической схемы Эль-Гамала^[11], где добавлено хеширование сообщения, а также все логарифмы вычисляются по ***mod q***, что позволяет сделать подпись короче по сравнению с аналогами ^[12]. На основе схемы Эль-Гамала построены и другие алгоритмы, например - российский ГОСТ 34.10-94, который сейчас считается устаревшим. На смену ему пришел стандарт ГОСТ Р 34.10-2012^[13], в котором используется группа точек эллиптической кривой.

Подобная модификация, т.е. переход от мультипликативной группы по модулю простого числа к группе точек эллиптической кривой существует и для DSA - ECDSA^[14] (англ. Elliptic Curve Digital Signature Algorithm - алгоритм цифровой подписи на эллиптических кривых). Он применяется, например, в криптовалюте bitcoin для подтверждения транзакций. Этот перевод позволяет уменьшить размер ключей без ущерба для безопасности - в системе bitcoin размер закрытого ключа 256 бит, а соответствующего ему открытого - 512 бит.

Другой распространенный алгоритм с открытым ключом (используется и для шифрования, и для цифровой подписи), RSA (назван в честь авторов: Ривест, Шамир, Адлеман), основан на сложности факторизации больших чисел.

Криптографическая стойкость

Любую атаку на алгоритм можно описать так: злоумышленник получает все открытые параметры подписи и некий набор пар (*сообщение*, *подпись*) и пытается, используя этот набор, создать действительную подпись для нового сообщения, не представленного в наборе.

Эти атаки можно условно разделить на две группы - во-первых, злоумышленник может попытаться восстановить секретный ключ x , и тогда он сразу получает возможность подписать любое сообщение, во-вторых, он может попробовать создать действительную подпись для нового сообщения без прямого восстановления секретного ключа.

Предсказуемость случайного параметра

Равномерное распределение случайного параметра k очень важно для безопасности системы. Если известны несколько последовательных бит параметра k для ряда подписей, то секретный ключ возможно восстановить с высокой вероятностью.^[15]

Повторение параметра для двух сообщений ведет к простому взлому системы. Это может произойти при использовании плохого генератора псевдослучайных чисел. Данная уязвимость в системе PlayStation 3 позволяла подписывать от имени Sony любые программы. В некоторых реализациях системы bitcoin для Android злоумышленник мог получить доступ к кошельку.^{[16][17]} В обоих примерах использовалась система ECDSA^[14].

Если для двух сообщений m_1, m_2 использовался один и тот же параметр k , тогда их подписи (r, s) будут иметь одинаковые r , но разные s , назовем их s_1, s_2 .

Из выражения для s можно выразить общий k :

$$k = (H(m) + xr)s^{-1} \mod q.$$

И приравнять общий k для разных сообщений:

$$(H(m_1) + xr)s_1^{-1} \mod q = (H(m_2) + xr)s_2^{-1} \mod q$$

Отсюда легко выразить секретный ключ x :

$$x = \frac{H(m_1)s_1^{-1} - H(m_2)s_2^{-1}}{r(s_2^{-1} - s_1^{-1})}$$

Existential forgery

На некоторые алгоритмы цифровой подписи возможна атака существующей подделки (Existential forgery). Она заключается в том, что для подписи (либо вообще случайной, либо созданной по некоторому правилу) возможно создать корректное сообщение (которое, правда, обычно не несет смысла), используя только открытые параметры.

Для схемы DSA подпись $r = g^e y \mod p \mod q$, $s = r$ при любом e корректна для сообщения с хешем $H(m) = es$.

Это одна из причин хеширования входного сообщения. При корректном выборе хеш-функции алгоритм DSA защищен от этой атаки, потому что обращение криптографической хеш-функции (т.е. для заданного k нахождение m такого, что $H(m) = k$) является вычислительно сложной задачей.^[18]

Восстановление ключа

условие корректности подписи можно переписать в ином виде:

$$g^{ks} \mod p \mod q = g^{H(m)+xr} \mod p \mod q$$

это уравнение эквивалентно (т.к. мультипликативный порядок g по модулю p равен q)

$$H(m) \mod q = ks - xr \mod q$$

значит можно считать, что злоумышленнику для восстановления ключа требуется решить систему уравнений вида

$$H(m_i) \mod q = k_i s_i - x r_i \mod q$$

но в этой системе неизвестен x и все k_i , значит число неизвестных на единицу больше, чем уравнений и при любом x найдутся k_i , удовлетворяющие системе. Так как q - большое простое число, то для восстановления $x \mod q$ потребуется экспоненциальное число пар (сообщение, подпись).^[1]

Подделка подписи

Можно попытаться подделать подпись, не зная секретный ключ, то есть попытаться решить уравнение

$$r^s \bmod p \bmod q = g^{H(m)} y^r \bmod p \bmod q$$

относительно r и s . При каждом фиксированном r уравнение эквивалентно вычислению дискретного логарифма.^[1]

Система проверки реализации алгоритма

Условия лицензии позволяют реализовывать алгоритм программно и аппаратно. НИСТ создал DSAVS^[19] (англ. *The Digital Signature Algorithm Validation System - система проверки алгоритма цифровой подписи*). DSAVS состоит из нескольких модулей проверки на соответствие стандарту, которые тестируют каждый компонент системы независимо от других. Тестируемые компоненты реализации:

1. Генерация доменных параметров
2. Проверка доменных параметров
3. Генерация пары открытый-закрытый ключ
4. Создание подписи
5. Проверка подписи

Для проверки реализации разработчик должен подать заявку на тестирование его реализации в CMT laboratory (англ. Cryptographic Module Testing Laboratory - лаборатория тестирования криптографических модулей).^[5]

Генерация простых чисел

При работе алгоритма DSA требуется два простых числа (p и q), следовательно необходим генератор простых или псевдопростых чисел.

Для генерации простых чисел используется алгоритм Шо-Тейлора^[20].

Псевдопростые числа генерируются с помощью хеш-функции и для проверки на простоту используется вероятностный тест Миллера — Рабина. К нему может добавляться одиночный тест простоты Люка.^[5]

Необходимое число итераций зависит от длины используемых чисел и от алгоритма проверки^[5]:

параметры	только М-Р тест	М-Р тест + тест Люка
р: 1024 бит q: 160 бит вероятность ошибки 2^{-80}	40	р: 3 q: 19
р: 2048 бит q: 224 бит вероятность ошибки 2^{-112}	56	р: 3 q: 24
р: 2048 бит q: 256 бит вероятность ошибки 2^{-112}	56	р: 3 q: 27
р: 3072 бит q: 256 бит вероятность ошибки 2^{-128}	64	р: 2 q: 27

Генерация случайных чисел

Для работы алгоритма требуется также генератор случайных или псевдослучайных чисел. Этот генератор нужен для создания частного пользовательского ключа x , а также для создания секретного случайного параметра k .

Стандарт предлагает различные способы генерации псевдослучайных чисел используя блочные шифры или хеш-функции.^{[5][21]}

Примечания

1. Patent US 5231668 A.
2. FIPS 186-1.
3. FIPS 186-2.
4. FIPS 186-3.
5. FIPS 186-4.
6. FIPS 180-4.
7. FIPS 202.
8. Finding Collisions in the Full SHA-1.
9. Freestart collision for full SHA-1.

10. [NIST Special Publication 800-57](#).
11. [Elgamal, 1985](#).
12. [C. P. Schnorr](#). Efficient Identification and Signatures for Smart Cards (https://link.springer.com/chapter/10.1007/0-387-34805-0_22) (англ.) // *Advances in Cryptology — CRYPTO' 89 Proceedings* / Gilles Brassard. — New York, NY: Springer, 1990. — P. 239–252. — ISBN 978-0-387-34805-6. — doi:10.1007/0-387-34805-0_22 (https://dx.doi.org/10.1007%2F0-387-34805-0_22). Архивировано (https://web.archive.org/web/20220412210143/https://link.springer.com/chapter/10.1007/0-387-34805-0_22) 12 апреля 2022 года.
13. [ГОСТ Р 34.11—2012](#) (<http://protect.gost.ru/document.aspx?control=7&id=180209>)
14. [The Elliptic Curve Digital Signature Algorithm \(ECDSA\)](#).
15. [The Insecurity of the Digital Signature Algorithm with Partially Known Nonces](#).
16. [ECDSA - Application and Implementation Failures](#).
17. [Elliptic Curve Cryptography in Practice](#).
18. [Security Arguments for Digital Signatures and Blind Signatures](#).
19. [The Digital Signature Algorithm Validation System](#).
20. [Generating strong primes](#).
21. [Random Number Generation](#).

Литература

Стандарты и патенты

- [FIPS. PUB 186-1](#) (<http://www-archive.mozilla.org/projects/security/pki/nss/fips1861.pdf>) (англ.).
- [FIPS. PUB 186-2](#) (<http://csrc.nist.gov/publications/fips/archive/fips186-2/fips186-2.pdf>) (англ.).
- [FIPS. PUB 186-3](#) (http://csrc.nist.gov/publications/fips/fips186-3/fips_186-3.pdf) (англ.).
- [FIPS. PUB 186-4](#) (<http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>) (англ.).
- [FIPS. PUB 180-4](#) (<http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>) (англ.). Архивировано (<https://web.archive.org/web/20161126003357/http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>) 26 ноября 2016 года.
- [FIPS. PUB 202](#) (<http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>) (англ.).
- [FIPS. PUB 202](#) (<http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>) (англ.).
- [David W. Kravitz](#). Digital signature algorithm 5231668 A (<https://www.google.com/patents/US5231668>) (англ.).
- [NIST Special Publication 800-57 Part 1 Revision 4](#). Recommendation for Key Management (<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf>) (англ.).
- [ГОСТ Р 34.10-2012](#). Информационные технологии. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи (<http://protect.gost.ru/document.aspx?control=7&id=180151>).
- [The Digital Signature Algorithm Validation System](#) (<http://csrc.nist.gov/groups/STM/cavp/documents/dss/DSAVS.pdf>) (англ.).

Статьи

- [Marc Stevens, Pierre Karpman, Thomas Peyrin](#). Freestart collision for full SHA-1 (<https://eprint.iacr.org/2015/967.pdf>) (англ.).
- [NIST Special Publication 800-90A](#) Recommendation for Random Number Generation Using Deterministic Random Bit Generators (<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-90Ar1.pdf>) (англ.).

- *J. Shawe-Taylor*. Generating strong primes (http://digital-library.theiet.org/content/journals/10.1049/el_19860598) (англ.).
- *Xiaoyun Wang, Yiqun Lisa, Hongbo Yu*. Finding Collisions in the Full SHA-1 (<http://people.csail.mit.edu/yiqun/SHA1AttackProceedingVersion.pdf>) (англ.).
- *Phong Q. Nguyen, Igor E. Shparlinski*. The Insecurity of the Digital Signature Algorithm with Partially Known Nonces (<https://link.springer.com/article/10.1007%2Fs00145-002-0021-3>) (англ.).
- *David Pointcheval, Jacques Stern*. Security Arguments for Digital Signatures and Blind Signatures (<http://www.math.uni-frankfurt.de/~dmst/teaching/SS2012/Vorlesung/Point.Stern.pdf>) (англ.).
- *Markus Schmid*. ECDSA - Application and Implementation Failures (<https://pdfs.semanticscholar.org/a7c7/97dab90b439b0d436ee8807db4e9207cff32.pdf>) (англ.).
- *Don Johnson, Alfred Menezes, Scott Vanstone*. The Elliptic Curve Digital Signature Algorithm (ECDSA) (<https://link.springer.com/article/10.1007/s102070100002>) (англ.).
- *Elgamal T.* A Public-Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms, A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms (<http://caislab.kaist.ac.kr/lecture/2010/spring/cs548/basic/B02.pdf>) (англ.) // *IEEE Transactions on Information Theory* / F. Kschischang — IEEE, 1985. — P. 10—18. — ISSN 0018-9448 (<https://www.worldcat.org/issn/0018-9448>); 1557-9654 (<https://www.worldcat.org/issn/1557-9654>) — doi:10.1109/TIT.1985.1057074 (<https://dx.doi.org/10.1109/TIT.1985.1057074>); doi:10.1007/3-540-39568-7_2 (https://dx.doi.org/10.1007/3-540-39568-7_2)
- *Joppe W. Bos, J. Alex Halderman, Nadia Heninger, Jonathan Moore, Michael Naehrig, Eric Wustrow*. Elliptic Curve Cryptography in Practice (<https://eprint.iacr.org/2013/734.pdf>) (англ.).

Источник — <https://ru.wikipedia.org/w/index.php?title=DSA&oldid=132410511>

Эта страница в последний раз была отредактирована 17 августа 2023 в 14:50.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)